République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Mohammed Seddik Benyahia – Jijel

Faculté des Sciences Exactes et de l'Informatique

Département d'Informatique

Cartographie d'une Fonction Mystère

Propagation avant, Rétropropagation et Optimisation sans framework

---

Réalisé par :
Foughalia Lina

Encadré par :
Dr. Mohamed Nadjib Zennir

Année universitaire : 2025 – 2026

Table des matières

1 Introduction **2**
1.1 Objectif du TP . . . 2
1.2 Problème posé . . . 2
1.3 Structure du projet . . . 2

2 Génération du Dataset **2**
2.1 Création des données . . . 2
2.2 Normalisation . . . 2
2.3 Visualisation de référence . . . 3

3 Architecture du Réseau **3**
3.1 Description des couches . . . 3
3.2 Initialisation des poids . . . 3
3.3 Activation ReLU et fonction de perte . . . 3

4 Rétropropagation **4**
4.1 Propagation avant . . . 4
4.2 Propagation arrière . . . 4
4.3 Implémentation . . . 4
4.4 Optimiseur SGD + Momentum . . . 4

5 Entraînement et Résultats **5**
5.1 Hyperparamètres . . . 5
5.2 Console d'entraînement . . . 5
5.3 Courbe d'apprentissage . . . 5
5.4 Comparaison MLP et fonction de référence . . . 6

6 Questions de Réflexion **6**
6.1 Q1 — Impact de l'activation : ReLU vs Linéaire . . . 6
6.2 Q2 — Capacité : architecture [2, 4, 1] . . . 7
6.3 Q3 — Impact du Momentum . . . 8

7 Conclusion **9**

1 Introduction

1.1 Objectif du TP

Ce travail pratique consiste à implémenter de zéro un **Perceptron Multicouche** (MLP) en utilisant uniquement la bibliothèque `NumPy`, sans recourir à des frameworks de haut niveau tels que PyTorch ou TensorFlow. Les trois composantes fondamentales sont codées manuellement :

— La **propagation avant** (*forward pass*)
— La **rétropropagation du gradient** (*backpropagation*)
— L'**optimisation** des paramètres par descente de gradient

1.2 Problème posé

Le réseau est entraîné à approximer la surface mathématique suivante, définie sur le domaine $[-5, 5]^2$:

$$f(x, y) \;=\; \sin\left(\sqrt{x^2 + y^2}\right) \;+\; 0{,}5 \cdot \cos(2x + 2y) \tag{1}$$

Cette fonction est la superposition de deux composantes :

— $\sin\left(\sqrt{x^2 + y^2}\right)$: anneaux concentriques autour de l'origine,
— $0{,}5 \cdot \cos(2x + 2y)$: ondulations diagonales à haute fréquence.

1.3 Structure du projet

Fichier	Rôle
`main.py`	Point d'entrée, orchestre les quatre étapes
`src/dataset.py`	Génération des données et normalisation
`src/model.py`	Classe MLP : forward, backprop, optimiseur
`src/train.py`	Boucle d'entraînement mini-batch
`src/visualize.py`	Comparaison visuelle MLP vs référence

2 Génération du Dataset

2.1 Création des données

Un ensemble de 2000 points (x, y) est tiré uniformément dans $[-5, 5]^2$, puis la valeur $z = f(x, y)$ est calculée selon l'équation 1.

Listing 1 – Generation du dataset (dataset.py)

```python
def generate_dataset(n_points=2000, x_min=-5.0, x_max=5.0, seed=42):
    rng = np.random.default_rng(seed)
    xy = rng.uniform(x_min, x_max, size=(n_points, 2))
    z = target_function(xy[:, 0], xy[:, 1]).reshape(-1, 1)
    return xy, z   # X : (2000, 2),  z : (2000, 1)
```

2.2 Normalisation

Les entrées X et les sorties z sont normalisées séparément vers $[-1, 1]$ par la transformation min-max. Sans cette étape, les gradients seraient déséquilibrés et la convergence instable.

$$x_{\text{norm}} \;=\; 2 \cdot \frac{x - x_{\min}}{x_{\max} - x_{\min}} - 1 \tag{2}$$

2.3 Visualisation de référence

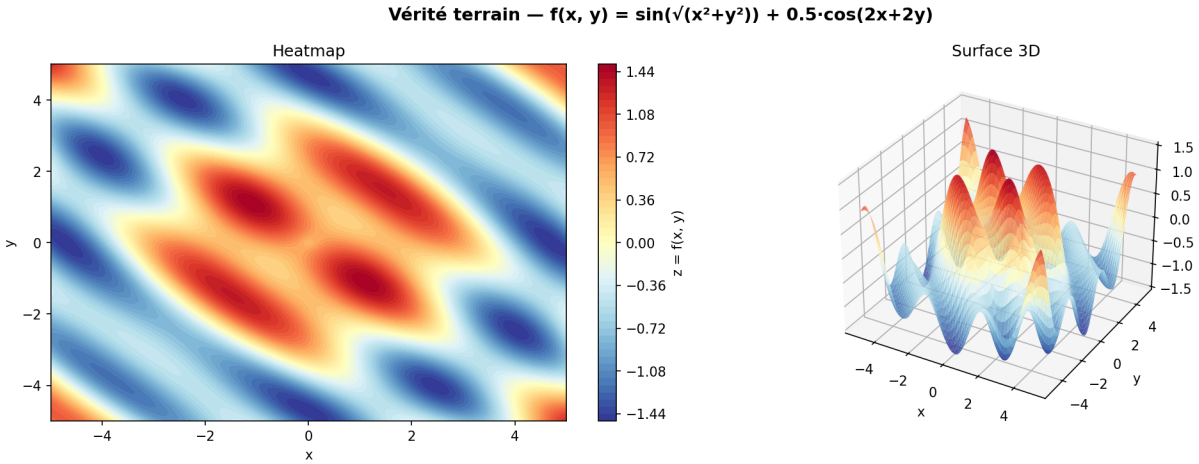

FIGURE 1 – Vérité terrain — heatmap et surface 3D de $f(x, y)$

La figure 1 illustre la complexité multi-fréquentielle de la fonction cible, justifiant le recours à un réseau non-linéaire.

3 Architecture du Réseau

3.1 Description des couches

L'architecture retenue est **[2, 64, 64, 1]** : deux entrées, deux couches cachées de 64 neurones, une sortie scalaire.

Couche	Neurones	Activation	Rôle
Entrée	2	—	Coordonnées (x, y)
Cachée 1	64	ReLU	Extraction de caractéristiques
Cachée 2	64	ReLU	Composition de caractéristiques
Sortie	1	Linéaire	Valeur $z = f(x, y)$
Paramètres totaux			4 417

3.2 Initialisation des poids

$$\text{He (couches ReLU)} : \quad W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}}\right) \tag{3}$$

$$\text{Xavier (sortie)} : \quad W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}} + n_{\text{out}}}\right) \tag{4}$$

3.3 Activation ReLU et fonction de perte

$$\text{ReLU}(z) = \max(0,\ z), \qquad \text{ReLU}'(z) = \mathbf{1}[z > 0].$$

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2, \qquad \frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{2}{N}(\hat{y} - y).$$

Sans activation non-linéaire, la composition de couches resterait une transformation affine ($W_3W_2W_1X = W_{\text{eq}}X$), incapable de représenter une surface sinusoïdale.

4 Rétropropagation

4.1 Propagation avant

$$Z_i = A_{i-1}W_i + b_i, \qquad A_i = \begin{cases} \mathrm{ReLU}(Z_i) & \text{couche cachée} \\ Z_i & \text{sortie} \end{cases}$$

4.2 Propagation arrière

Quantité	**Formule**
Gradient de la perte	$dA_{\text{out}} = \frac{2}{N}(\hat{y} - y)$
Gradient Z_i (cachée)	$dZ_i = dA_i \odot \mathrm{ReLU}'(Z_i)$
Gradient Z_i (sortie)	$dZ_i = dA_i$
Gradient des poids	$dW_i = A_{i-1}^\top dZ_i$
Gradient des biais	$db_i = \mathbf{1}^\top dZ_i$
Propagation arrière	$dA_{i-1} = dZ_i W_i^\top$

Toutes ces opérations sont **vectorisées** : elles portent sur le mini-batch entier sans aucune boucle sur les exemples individuels.

4.3 Implémentation

Listing 2 – Retropropagation vectorisee (model.py)

```python
def backward(self, y_true):
    Zs, As = self._cache["Zs"], self._cache["As"]
    dW = [None] * self.n_layers
    db = [None] * self.n_layers
    dA = 2.0 * (As[-1] - y_true) / y_true.shape[0]
    for i in reversed(range(self.n_layers)):
        if i == self.n_layers - 1:
            dZ = dA
        else:
            dZ = dA * (Zs[i] > 0)
        dW[i] = As[i].T @ dZ
        db[i] = dZ.sum(axis=0, keepdims=True)
        dA    = dZ @ self.W[i].T
    return {"dW": dW, "db": db}
```

4.4 Optimiseur SGD + Momentum

$$v \leftarrow \beta v + (1-\beta)\nabla\mathcal{L}, \qquad W \leftarrow W - \alpha\, v, \qquad \alpha = 8\times 10^{-3}, \quad \beta = 0{,}9.$$

5 Entraînement et Résultats

5.1 Hyperparamètres

Hyperparamètre	Valeur	Justification
Architecture	[2, 64, 64, 1]	Capacité
Époques	1000	Maximum autorisé
Mini-batch	32	62 000 mises à jour par entraînement
Learning rate	8×10^{-3}	Optimal avec momentum = 0,9
Momentum	0,9	Accélère la convergence
Normalisation	Min-Max $[-1, 1]$	Équilibre les échelles

5.2 Console d'entraînement

Listing 3 – Sortie lors de l'execution

```
Epoque  100/1000  |  Loss MSE : 0.101561
Epoque  200/1000  |  Loss MSE : 0.070938
Epoque  300/1000  |  Loss MSE : 0.045757
Epoque  400/1000  |  Loss MSE : 0.027856
Epoque  500/1000  |  Loss MSE : 0.019758
Epoque  600/1000  |  Loss MSE : 0.014437
Epoque  700/1000  |  Loss MSE : 0.012249
Epoque  800/1000  |  Loss MSE : 0.010558
Epoque  900/1000  |  Loss MSE : 0.009671
Epoque 1000/1000  |  Loss MSE : 0.009050
Loss initiale : 0.271631  ->  Loss finale : 0.009050  (x30)
MSE test : 0.023124   |   MAE test : 0.111134
```

5.3 Courbe d'apprentissage

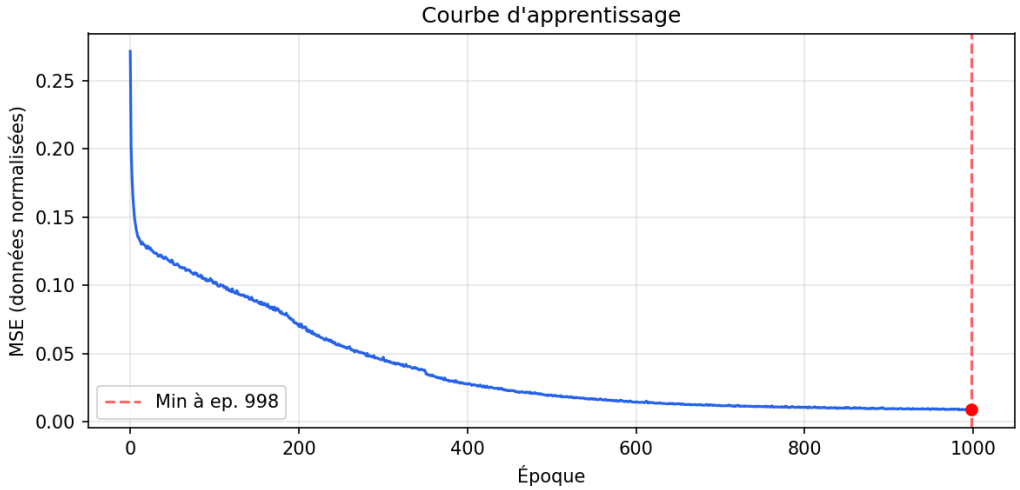

FIGURE 2 – Évolution de la perte MSE sur 1000 époques (réduction ×30)

5.4 Comparaison MLP et fonction de référence

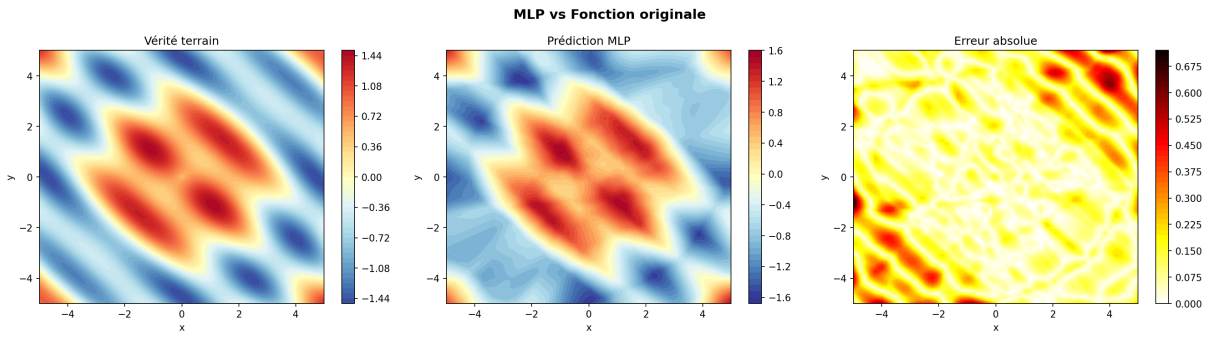

FIGURE 3 – Heatmaps : vérité terrain, prédiction MLP et erreur absolue

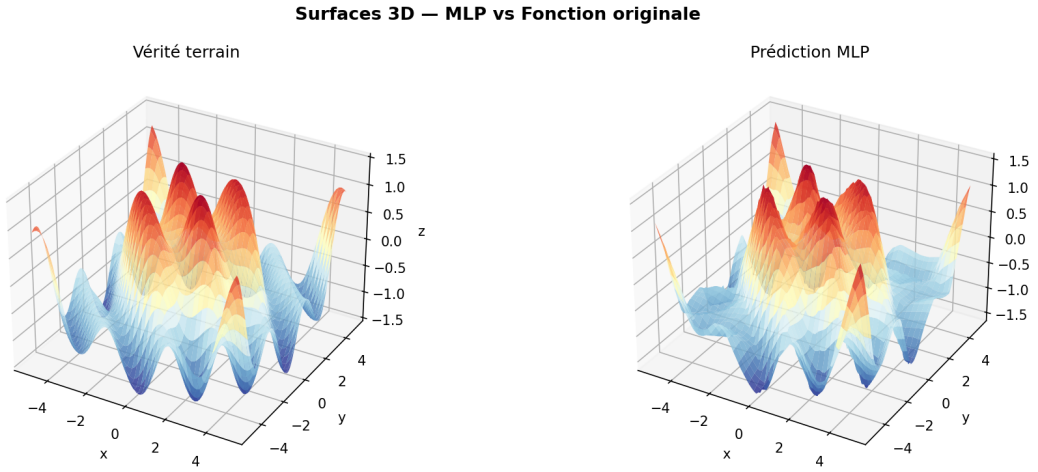

FIGURE 4 – Surfaces 3D : vérité terrain (gauche) et prédiction MLP (droite)

Métrique	**Valeur**	**Interprétation**
Loss finale	0,009	Convergence satisfaisante
MSE test	0,023	Bonne généralisation
MAE test	0,111	Erreur moyenne faible
Réduction loss	$\times 30$	De 0,272 à 0,009
Temps d'exécution	66 s	CPU standard

6 Questions de Réflexion

6.1 Q1 — Impact de l'activation : ReLU vs Linéaire

Si toutes les fonctions ReLU sont remplacées par des fonctions linéaires, la composition des couches reste une transformation affine unique :

$$W_3\big(W_2(W_1X + b_1) + b_2\big) + b_3 = W_{\text{eq}}X + b_{\text{eq}}.$$

Le réseau se réduit à une régression linéaire (loss $\approx$ 0,35) incapable de représenter la surface sinusoïdale. **La non-linéarité est indispensable.**

6.2 Q2 — Capacité : architecture [2, 4, 1]

L'expérience a été réalisée avec les mêmes hyperparamètres (lr $= 8 \times 10^{-3}$, momentum $= 0{,}9$, 1000 époques, batch $= 32$) en changeant uniquement l'architecture.

Architecture	Paramètres	Loss finale	MSE test	Réduction
[2, 64, 64, 1]	4 417	0,009	**0,023**	×30
[2, 4, 1]	17	0,124	0,285	×2,1

Le réseau [2, 4, 1] stagne dès l'époque 765 avec une loss de 0,124, soit **14 fois plus élevée** que l'architecture recommandée. Avec seulement 4 neurones, le réseau ne dispose que de 4 hyperplans pour découper $[-5, 5]^2$, alors que la fonction présente une quinzaine d'oscillations. **La capacité du réseau doit être proportionnelle à la complexité de la cible.**

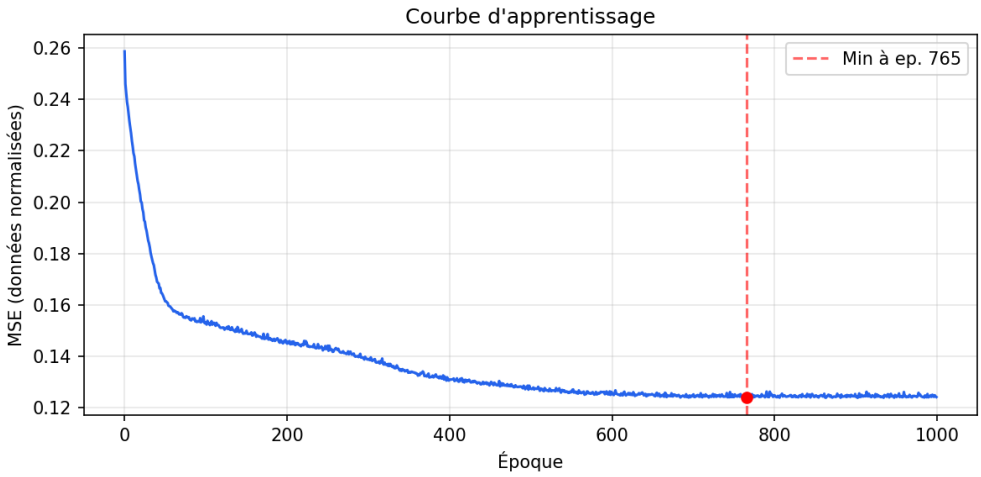

FIGURE 5 – [2, 4, 1] — Courbe de loss : stagnation dès l'époque 765, minimum à 0,124

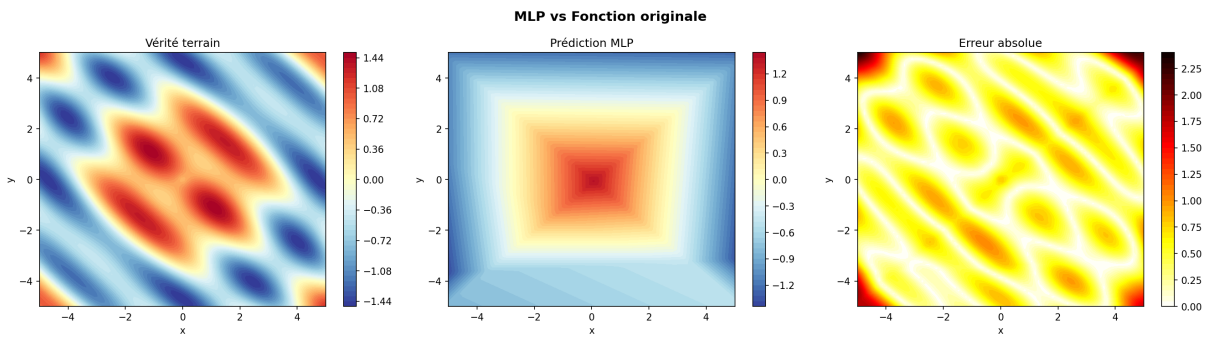

FIGURE 6 – [2, 4, 1] — La prédiction (centre) est une surface lisse sans oscillation ; l'erreur absolue (droite) couvre l'intégralité du domaine

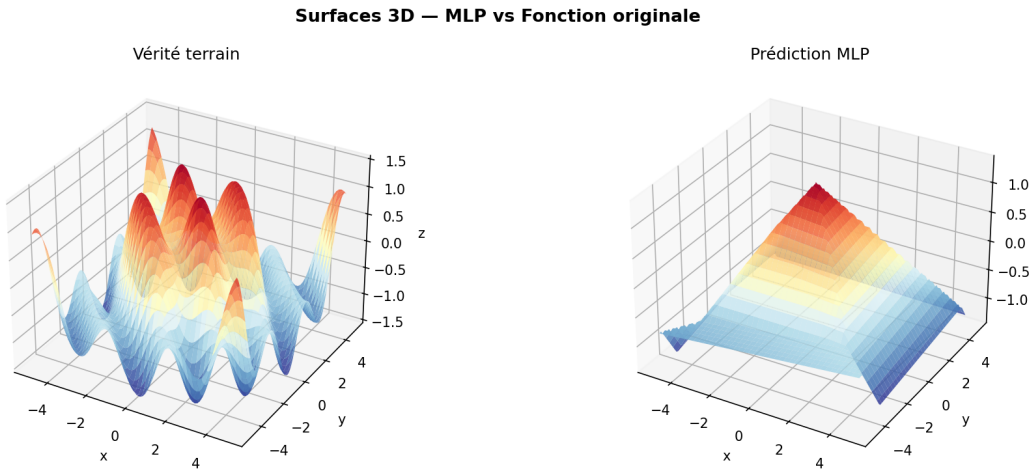

FIGURE 7 – $[2, 4, 1]$ — Surface 3D prédite : simple tente conique, aucun détail de f capturé

6.3 Q3 — Impact du Momentum

L'expérience compare SGD pur ($\beta = 0$) et SGD + Momentum ($\beta = 0{,}9$) sur la même architecture [2, 64, 64, 1] avec lr $= 8 \times 10^{-3}$.

Optimiseur	Loss finale	MSE test	MAE test	Réduction
SGD pur ($\beta = 0$)	0,011	0,037	0,137	$\times$22,6
SGD + Momentum ($\beta = 0{,}9$)	**0,009**	**0,023**	**0,111**	$\times$**30,0**

Le SGD pur converge correctement mais reste bloqué à une MSE test de 0,037 contre 0,023 avec le Momentum, soit **38 % de moins**. La courbe (figure 8) montre un léger rebond après l'époque 988, signe d'oscillations que le Momentum élimine grâce à son effet de lissage.

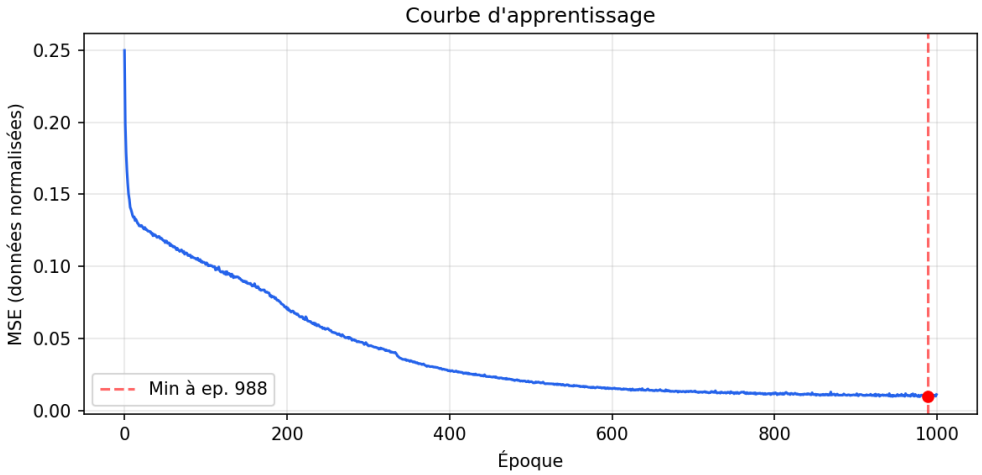

FIGURE 8 – SGD pur ($\beta = 0$) — minimum atteint à l'époque 988, puis rebond léger visible

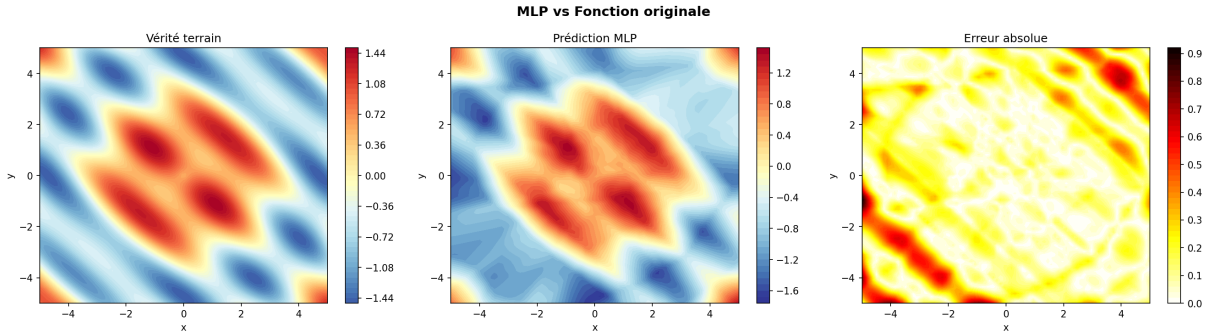

FIGURE 9 – SGD pur — les structures principales sont capturées mais l'erreur absolue reste plus élevée qu'avec Momentum, en particulier aux bords

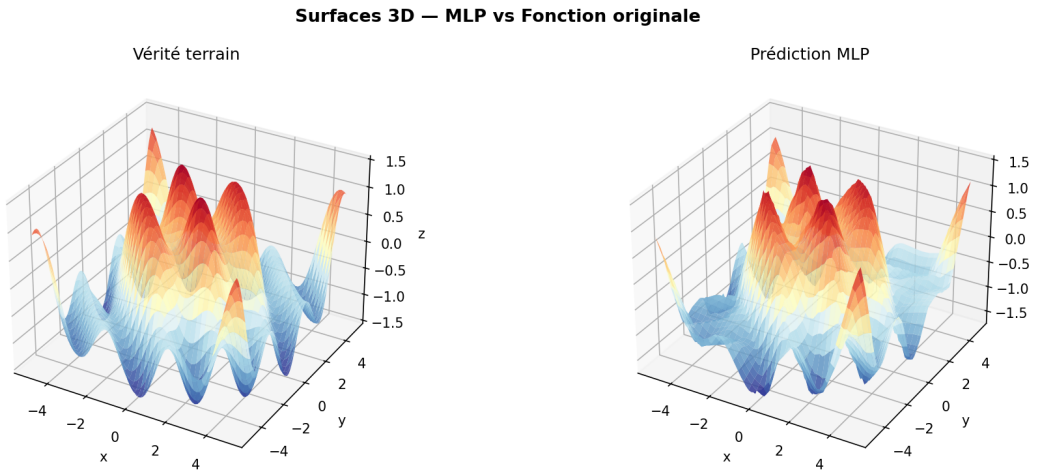

FIGURE 10 – SGD pur — surface 3D prédite : oscillations visibles mais moins fidèles qu'avec Momentum

Le Momentum accumule de l'inertie dans les directions stables, atténue les oscillations et franchit les plateaux de faible gradient, améliorant la MSE test d'un facteur $\times 1{,}6$.

7 Conclusion

Ce TP démontre qu'un MLP implémenté avec **NumPy uniquement** est capable d'approximer une surface mathématique bi-fréquentielle complexe avec une précision satisfaisante (MSE test $= 0{,}023$, réduction $\times 30$). Les expériences de réflexion ont confirmé quantitativement trois points clés : l'architecture [2, 4, 1] atteint une MSE test de 0,285 contre 0,023 pour [2, 64, 64, 1] (facteur $\times 12$) ; le SGD pur donne une MSE test de 0,037 contre 0,023 avec Momentum (amélioration de 38 %).

Références

[1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org

[2] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

[3] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers. *ICCV 2015*.